

Adding scores and other functionality

Now, let's move on to the finishing touches and give the game more features. One thing we can add easily is some sort of score mechanism to introduce the competitive spirit into the game.

To add up the score, we can easily add a method to the player class and also add a way to increase the speed of the game as the player progresses through it, both of which can be called in the main game's 'if' block.

```
def update(self):
    self.score += ( (frameCount/
frameCount) * 0.05)
    if self.highscore < self.score:
        self.highscore = self.score
        self.speed += (
(frameCount/frameCount) * 0.005)
```

Now, for the proper usage of the above function, we add it to *draw()* as shown in the code segment below:

```
def draw():
    #globals
    If gameOver == False:
        frameRate( int( pl.speed) )
        perspective()
            camera()
            textSize(14)
            textAlign(LEFT)
            fill(0)
            text("Press P to Pause",
0, 15)
            text("Press R to Resume", 0, 40)

            textAlign(RIGHT)
            text(int(pl.score), width, 15)
            text(int(pl.highscore), width, 40)
            text("Score: ", width-30, 15)
            text("High Score: ", width-30, 40)

            #previous code

            pl.update()
    else:
        #previous code
```

When that last addition gets executed, your game will resemble

Figure 6 and the 'Game Over' screen will be like what's shown in Figure 7.

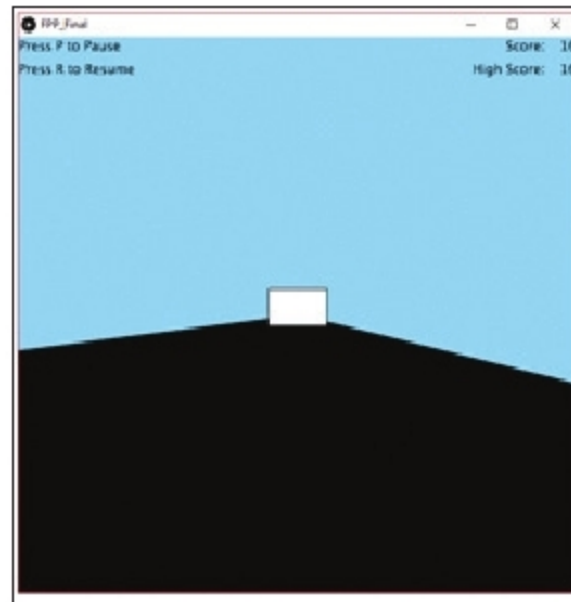


Figure 6: Score



Figure 7: Updated 'Game Over'screen

And with that, you have successfully made a 3D game! And if this is your maiden effort in the field of 3D programming, rejoice for your efforts have paid off. You can now proudly say you have dabbled in the world of 3D graphical programming.

The full code for the game

```
gameOver = False
class player:
    def __init__(self):
        self.j_flag = False
        self.grav_flag = True
        self.eyeY = 1.5
        self.ptY = 1.5
        self.fov = PI/3
        self.speed = 40
        self.flag = 1
        self.score = 0
        self.highscore = 0
```

```
def jump(self):
    self.eyeY += 0.1
    self.ptY += 0.05

def grav(self):
    self.eyeY -= 0.1
    self.ptY -= 0.05

def reset(self):
    self.j_flag = False
    self.grav_flag = True

def update(self):
    self.score += ( (frameCount/
frameCount) * 0.05)
    if self.highscore < self.score:
        self.highscore = self.score
        self.speed += ( (frameCount/
frameCount) * 0.005)
class obstacle:
    def __init__(self, z, ht):
        self.z = z
        self.ht = ht

    def move(self):
        self.z += 10

    def check_bounds(self):
        if self.z > 100:
            self.z = -500*10
            self.ht = random(100, 150)

def setup():
    global pl, obs, base_ht, textArea
    size(600, 600, P3D)
    base_ht = height-100
    pl = player()

    obs = []
    z_val = -500*10
    for i in range(6):
        o = obstacle(z_val, random(100,
150))
        obs.append(o)
        z_val -= 850
        textMode(SHAPE)

def draw():
    global pl, obs, base_ht, gameOver,
textArea
    if gameOver == False:
        background(137, 207, 240)
```